

Weather Risk Intelligence Agent

We are a logistics company operating regional distribution hubs across the US.

Severe weather shuts hubs down, delays shipments and costs us money. Each year we choose a handful of hubs to invest in resilience upgrades.

We are exploring how AI agents can help our analysts decide which hubs are most exposed to weather disruption and should be prioritized for investment.

Build an AI-powered agent that can answer questions such as:

- Which hubs in the Midwest are most exposed to winter disruption?
- Compare Miami and Houston in terms of hurricane and flood exposure.
- What percentage of days in Denver last year had snowfall?
- Why is the Dallas hub's weather disruption risk high?

The agent should:

- Use public APIs to gather weather and hazard data
- Rank or compare hubs based on your defined logic or KPI
- Explain its reasoning clearly
- Support conversational follow-up questions

Requirements

- Include some deterministic scoring or ranking logic (not only LLM output)
- Include a chat interface to talk with the agent (voice is a bonus)
- Bonus: a scheduled or webhook-triggered alert when a hub's risk score changes
- Expose the agent through an API that the chat interface uses
- Enforce a structured output format (JSON schema) between the LLM and your code
- Include a small evaluation set and a way to run it
- Clearly communicate assumptions, uncertainty and scoping

Deliverables - within 24 hours:

- Source code, with instructions to run it. You should be able to run the agent live during the follow-up interview. Deploying it to a URL is preferred, but not mandatory.
- Short design/architecture document explaining:
 - System architecture: the components and how they talk to each other
 - Repository structure
 - Data storage choice
- **The full session with the AI agent you worked with: the plan, and everything else relevant to your conversation with the agent. This is mandatory.**

This assignment is intended to be achievable in approximately one day. Prioritize clarity, reasoning and thoughtful design over completeness or polish. You may use any language, framework and LLM provider; Python is preferred. A narrow scope done well is better than a broad scope done shallowly.